



platformeconomics.ai – Voice AI Platform

Technical Plan

Introduction

We are building **platformeconomics.ai**, a next-generation voice AI platform for businesses. Acting as CEO/Architect (à la Elon Musk visionary style), our goal is to **rapidly deliver a voice-agent platform** by assembling best-in-class components. By leveraging Vapi's voice AI infrastructure, Cloudbeds' hospitality API, and modern web tools, we focus development on high-impact features that elevate customer experience, streamline operations, and drive revenue. This document outlines the technical architecture and development plan to launch our MVP by **November 12, 2025**, highlighting key platform components and multiple specialized AI agents working in concert.

Architecture Overview

Our platform is composed of integrated layers: a telephony/voice AI layer, an application logic layer, and a data integration layer. At a high level, the architecture includes:

- **Voice AI Engine (Vapi Platform):** Vapi provides the core voice assistant capabilities – telephony integration, speech recognition, and a conversation engine powered by large language models (LLMs). We will use Vapi's **Squads** feature to orchestrate multiple specialized AI assistants in one call ¹, and Vapi's function-calling **Tools** to enable external actions ². Vapi will manage inbound/outbound calls, real-time audio, and maintain conversation context across agent transfers.
- **Cloudbeds API Integration:** Cloudbeds is the hospitality management system we're targeting first (e.g. hotel reservations, guest info). We will integrate Cloudbeds' **Open API (v1.3)** to allow our voice agents to **query and update hotel data**. Cloudbeds offers 50+ endpoints covering guest info, reservations, etc., with a published OpenAPI spec ³. We'll generate **custom Vapi tools** corresponding to Cloudbeds API calls (for example: check room availability, create booking, modify reservation, fetch guest profile). These tools will be invoked by the voice agents as needed during conversations.
- **Backend Orchestrator (Node.js & Supabase):** A Node.js backend (with Supabase) will coordinate the logic that falls outside Vapi's direct scope. This includes hosting webhook endpoints for Vapi tools, handling authentication and data processing for Cloudbeds API calls, and managing user verification workflows. **Supabase Edge Functions** (Deno runtime) will be used to deploy lightweight serverless functions at the network edge for low-latency webhooks and verification links. Supabase will also provide a Postgres database for persistent data (e.g. storing user verification status, logs, etc.) and built-in auth (for magic link emails).
- **User Interface (shadcn/UI & Tailwind):** While the primary user interaction is via voice calls, we will build a minimal **web UI** for administration and to support certain user interactions. The UI (using ShadCN UI components and Tailwind CSS) will serve: (a) **Web Forms** for data collection (accessed via magic links during calls), and (b) an internal dashboard for our team to monitor agents and

configure settings. This modern React/Tailwind UI ensures rapid development and a clean, responsive design.

- **AI & Voice Services:** We will integrate **OpenAI** and **ElevenLabs** for the brains and voice of our agents. Vapi allows bringing your own models and APIs ⁴ – we'll use OpenAI's GPT-4 (or GPT-3.5) as the conversational model powering the voice assistants' understanding and reasoning. For text-to-speech, we'll integrate **ElevenLabs** voices for natural, human-like speech output (ElevenLabs is among Vapi's supported providers ⁴). Speech-to-text (transcription) can use Vapi's default (which may leverage providers like Deepgram or Whisper) to convert caller audio into text. This combination gives us state-of-the-art language understanding and voice quality with minimal custom engineering.

Architecture Diagram: (Conceptual) The voice flow begins when a customer calls a business's number. The call is routed to Vapi, which invokes our **Squad** of assistants. The first assistant (Concierge) engages the caller and potentially transfers control to other assistants (Booking, Support, Manager) as needed ¹. During the conversation, the assistants may call **custom Tools** to perform actions – e.g., querying Cloudbeds via our backend, sending a verification SMS, or launching a web form ⁵ ⁶. These tool calls hit our Node/Supabase backend (via HTTPS webhooks), which then interacts with Cloudbeds or other APIs and returns results to Vapi. Global context (like customer name, ID, reservation details) is maintained across the squad so each agent has a shared understanding. Finally, after the call, results and any collected structured data can be logged or sent to a webhook for record-keeping ⁷.

Key Platform Features and Components

1. Customer Identification (Global Parameters & Data Mapping)

Identifying the caller and linking them to customer records is critical for personalization. We will implement a **global caller profile** that is accessible to all agents in the squad. Key steps:

- **Caller Data Model:** Define a standard data structure for customer info – e.g. `caller_id`, name, company, loyalty status, current reservation details, verification status, communication preferences, etc. This serves as a **shared memory** for the call.
- **Global Variables:** Utilize Vapi's dynamic variables mechanism to pass and update this data during the call. Vapi allows using `{{double_curly_braces}}` in prompts to inject variable values ⁸. Our Node backend will start each call with known data (if available) by including `variableValues` in the API request that initiates the squad ⁸. For example, if the phone number matches a guest in Cloudbeds, we populate their name and reservation ID as variables so the assistant can greet them personally ("Hello, `{{firstName}}`...").
- **API Mapping:** We will map our global variables to Cloudbeds API fields. For instance, `caller_id` might correspond to a Cloudbeds guest ID, or we might use phone/email to look up the guest via Cloudbeds API at call start. A **lookup Tool** will query Cloudbeds (by phone or email) to retrieve customer profile and active bookings when the call begins, updating the global context for use by all agents.
- **Persistent Identification:** If the caller is new (not in Cloudbeds), the Concierge agent will gather basic info (name, etc.) and create a new guest record via API. Subsequent assistants can then reference this info. By standardizing how data is stored and accessed, each specialized agent can seamlessly pick up context (e.g. the Booking agent knows which guest or reservation it's handling because that was set by the Concierge agent).

2. Customer Verification (Magic Link & Code Authentication)

For sensitive operations or account-specific info, we will verify the caller's identity using one-time codes or magic links, ensuring security and building trust:

- **Multi-Channel Verification:** The assistant can send a one-time verification code via SMS to the caller's phone or a magic login link via email. We'll implement both methods to accommodate user preference. For phone verification, we'll use a **Supabase function or Twilio integration** to send an SMS with a code. For email, we can leverage Supabase Auth's magic link capability or send a custom email with a secure link.
- **Voice Workflow:** When verification is required, the voice agent will say: "I've sent a verification code to your phone/email. Please read it back to me." The caller provides the code, which the agent validates via a **VerifyCode Tool** calling our backend. If using a magic link (for smartphone users), the agent might instruct the caller to click the link; our backend (Supabase) will mark the verification status, and the agent can periodically check a **VerifyStatus Tool** until confirmation.
- **Integration with Global Profile:** The verification status (e.g. a boolean `isVerified`) will be stored in the global variables. Before any high-privilege action (like retrieving personal bookings or making changes), agents will ensure `isVerified == true`. If not, they invoke the verification flow. Once verified, all subsequent assistants in the squad are aware via the shared context. This prevents repeating the verification with each transfer.
- **Security Best Practices:** Codes will be time-limited and single-use. Magic links will expire quickly and be tied to the specific action. We will use **Supabase Edge Functions** to generate and validate these tokens securely. The entire verification process happens within the ongoing call, maintaining conversational continuity (the caller doesn't need to hang up).

3. Data Integration – Cloudbeds API Tools

Data integration is a cornerstone of our platform, allowing voice agents to actually **do useful work** (retrieve info and perform transactions). For the hospitality use-case, we target Cloudbeds (a popular hotel PMS). Our approach:

- **API Specification Import:** Cloudbeds provides OpenAPI specifications (e.g. on GitHub and Postman) for their REST API ⁹. We will import the latest **v1.3 spec** and analyze the available endpoints. Key ones likely include: guest profile lookup, room availability search, create/update reservation, cancel reservation, check-in/out operations, etc.
- **Tool Generation:** Using the API spec, we will programmatically generate Vapi **Custom Tools** for relevant endpoints. Each tool will encapsulate a single Cloudbeds API call. For example, a `findReservation` tool (to GET a reservation by ID), a `createReservation` tool (to POST a new booking), `listAvailableRooms` tool, etc. Each tool will have a **function name** and define the required input properties (parameters) and expected response. We can script this in Node.js: parse the OpenAPI YAML/JSON, and use Vapi CLI or API to create each tool entry. This saves time and ensures consistency with Cloudbeds' spec.
- **Authentication:** Cloudbeds API uses API keys or OAuth (per their docs). We will set up secure storage of API credentials (likely in Supabase or an env file) and have our tool webhook calls include the necessary auth headers. For example, our backend might maintain an OAuth token and attach it when forwarding the request. Vapi's custom tools allow setting a target URL – we'll use our

Supabase function URL as the tool's endpoint, and that function will handle calling Cloudbeds (this way we keep API keys secure server-side, not exposed in Vapi config).

- **Testing & Validation:** We will test each generated tool against the Cloudbeds sandbox using known test data. This involves using the Cloudbeds Postman collection (v1.3) to verify correct request/response formats. We'll deploy the tools in a dev environment on Vapi and run sample calls (using Vapi's CLI or simulator) to ensure the voice agent can successfully call the tools and handle responses. For example, the Booking agent asking "Do we have any rooms for June 10-12?" should trigger the `listAvailableRooms` tool and receive data to answer the caller.
- **Tool Use in Prompts:** In each assistant's prompt, we will provide instructions on when and how to use the tools by name. Vapi requires the assistant to be *explicitly told* to call a tool ¹⁰ ¹¹ . For instance, the Booking agent's system prompt will include: *"When the user asks to make a reservation or check availability, use the 'search-availability' tool to fetch open rooms, then present options."* This ensures the LLM knows to invoke the function rather than hallucinate.

By **automating Cloudbeds integration**, our platform gains powerful capabilities out-of-the-box to automate hotel operations with voice. The pattern we establish (importing API specs, generating tools) can later be replicated for other industries and APIs.

4. Web Links for Forms & Webhook Confirmations

Sometimes a voice call alone isn't sufficient to gather or present information – for example, collecting a detailed form (multiple fields), obtaining an e-signature, or showing visual content (like a menu or a document). Our platform introduces **Web Links** as a solution: these are secure URLs that the voice assistant can generate and send to the user (via SMS or email) during a call. They allow a seamless transition to a visual interface when needed, without losing the voice conversation thread. Key components:

- **Form Generation:** We will create a module to dynamically generate web forms or pages based on the assistant's needs. For example, if a hotel guest needs to fill out a check-in form or provide credit card authorization, the assistant will say "I'm sending a link to a form now." The backend (Supabase function) will create a unique one-time URL (perhaps on a subdomain of platformeconomics.ai) with an HTML form (built with our ShadCN/Tailwind UI components for consistency). We can pre-fill known info and include fields for whatever additional input is required. The form might include interactive elements: text inputs, upload buttons (for documents like an ID photo), or even a signature pad component.
- **Link Delivery:** The assistant will deliver the link to the user through an out-of-band channel. If the caller is on a mobile phone, the easiest is SMS. We can leverage Twilio (which Vapi likely uses under the hood for telephony) or another SMS API to send the link to the user's number. Alternatively, if we only have email, we can email the link (less immediate, but an option). The voice agent will confirm: "I've sent a link to your phone. Please open it to complete the form while we stay on the line."
- **Continuous Voice Guidance:** Importantly, the voice call remains connected while the user interacts with the web link on their device. The agent might play on-hold music or periodically check in ("Let me know when you're done with the form, or if you need help."). The link page can be kept simple with instructions since the voice agent is providing guidance in real time.
- **Webhook Confirmation:** Once the user submits the form (or completes the web interaction), our web app will trigger a **Webhook callback** to Vapi or our backend to signal completion. For instance, the form submission triggers a Supabase function that updates a status, and our voice agent can poll via a Tool (or we utilize Vapi's event hooks) to detect that the form is done. The content the user provided is then available to the assistant. The voice agent can resume the conversation, confirm

receipt ("Great, I've got your information."), and proceed with the next step. This mechanism ensures the **voice agent has real-time awareness** of the user's actions on the web link.

- **Use Cases:** Web links will be used for scenarios like: **Collecting detailed info** (e.g. a long survey or booking details that are faster to type than speak), **Document upload** (e.g. uploading a photo of ID or a document), **Displaying rich information** (like showing available room photos/rates or a confirmation summary for review), and **Signatures** (e.g. signing a rental agreement). The flexibility of having both voice and web interfaces working together greatly enhances user experience.

Technically, storing these links on Vapi was mentioned – possibly we will store a reference or token in a global variable or Vapi's context. The actual web content will be hosted on our side for full control. Each link will embed an ID that ties it to the call session or user, so when submitted we know which call/user to notify. This feature is a **differentiator**, blending voice AI with a visual channel for complex interactions.

5. Dynamic Context Generation for Conversations

To deliver a truly personalized, "feels human" experience, our agents will generate dynamic context and nuanced behavior based on the situation. We will leverage available data (from the customer profile and external APIs) to tailor the conversation flow in real time. Here's how:

- **Contextual Prompts:** Each assistant's prompt will be enhanced with real-time context variables. For example, if the Cloudbeds API indicates the caller is a VIP guest or has an upcoming birthday, the agent's prompt can include that information (e.g. *"The user is a Gold Member and is staying next week"*), so the assistant responds accordingly (*"Welcome back! Thank you for being a Gold Member with us."*). We will design **prompt templates** that incorporate variables like `{{guestStatus}}`, `{{reservationDates}}`, `{{lastInteraction}}` etc., which get filled at runtime ⁸.
- **Node-Specific Context:** In a Vapi squad, each assistant can represent a "node" of the conversation (a specific phase or task). We will generate context for each node. For instance, **Booking agent** context may include current availability or pricing retrieved just moments ago, whereas **Customer Support agent** context may include recent support ticket info or the guest's current room number. Our backend can fetch these details via API tools and then either feed them as variables or include them in the assistant's next prompt. This ensures the AI has all relevant facts before formulating a response.
- **Knowledge Base & Memory:** Besides transactional data, we might incorporate a small knowledge base relevant to each agent's domain (for example, policies for the concierge, FAQs for customer experience). Vapi supports knowledge base query tools (vector search, etc.), but given our timeline we may rely on prompt-based knowledge injection. We'll maintain a repository of up-to-date info (like hotel amenities, policies, etc.) that could be appended to prompts if needed.
- **Maintaining Continuity:** As the call progresses through multiple agents, **full conversation context is preserved** by Vapi's squad design ¹². The user should not have to repeat information. We will verify that any important details captured by one agent (structured data like name, dates, issue description) are passed along. This can be done via the global variables or using Vapi's `structuredDataMulti` feature which aggregates data from all assistants ⁷. By the time, say, the Virtual GM agent takes over a call, it will have a summary of everything discussed so far and can address the user appropriately (*"I understand you've been discussing an issue with your room's AC..."*).
- **Context Generation Examples:** If a caller says *"I stayed there last month"*, the agent can quietly call a Cloudbeds tool to pull up past stays, then weave that into the conversation (*"I see you stayed with us in September. Welcome back!"*). If a user is filling a web form (as per feature #4), the context might

include a note like “*User is currently uploading their ID via web link*” so the agent knows to wait or acknowledge once done. We will iteratively refine these dynamic prompt insertions during testing to maximize the naturalness and accuracy of the AI’s responses.

By smartly utilizing **contextual data and dynamic prompts**, we expect a significant improvement in the **quality of interactions** – making the voice agent more competent and personable than a one-size-fits-all bot.

6. No-Code Process & Vapi CLI DevOps

In order to move fast without breaking things, we will adopt a robust development workflow using the Vapi MCP CLI and best practices for version control. Our approach treats the voice assistants’ configurations as code, enabling safe iterations:

- **Infrastructure as Code:** We will use the **Vapi CLI** (MCP – Model-Connected Platform) to pull down the JSON/YAML definitions of our assistants, tools, and squads. Each change (adding a tool, tweaking a prompt, etc.) will be done via code and CLI commands rather than only the web GUI. This gives us the ability to diff, version, and rollback changes easily. All configuration files will be stored in our Git repository.
- **Backup & Patch Strategy:** Before modifying any assistant or tool, we will download the current config (creating a backup). Changes will be applied by patching only the relevant sections of the config. **Avoid full overwrite:** Vapi’s API requires including the full model object on PATCH, otherwise you risk wiping other settings ¹¹. We will build scripts to merge new changes into the existing config JSON to ensure nothing important is inadvertently lost. For example, when adding a new tool via API call, we’ll include the existing tool IDs and just append the new one, as the documentation warns that omitting fields overwrites them ¹³.
- **Automated Deployments:** Using Node.js, we can automate the sequence of steps for deploying updates. For instance, when we update an assistant’s prompt or add a transfer, we run a CLI command (or API call) to apply it on Vapi, then run a quick smoke test (via Vapi’s testing tools or a simulated call) to verify the change. This continuous integration approach catches errors early. We might set up a small script to run nightly regression tests using Vapi’s **automated testing** features (like simulated calls with expected outcomes) ¹⁴ so we ensure new changes don’t break existing functionality.
- **No-Code Philosophy:** Although we are writing code to configure the platform, the end goal is a sort of *no-code experience* for the platform’s operation. Once our system is in place, adding a new hotel or new industry integration should be doable with minimal coding – ideally just configuration and using our existing integration framework. The Vapi MCP CLI enables treating conversation flows as modular pieces that can be cloned and adjusted quickly. We will **not bypass Vapi’s best practices** – e.g., we won’t manually edit files on the server or hack around the system. Instead, we use supported methods (CLI, API, dashboard) to ensure maintainability.
- **Dev/Prod Environments:** We’ll maintain separate Vapi environments or at least separate sets of assistants for development vs production (leveraging Vapi’s enterprise features if available ¹⁵). All risky changes get tested with internal phone numbers first. Only once validated will we deploy to the live assistants (targeting our November 12 launch for a stable v1.0). Should any issue arise in production, we can quickly roll back to a previous config snapshot thanks to our version-controlled approach.

By combining a disciplined CLI-driven workflow with backup and patch procedures, we mitigate the chaos that often comes with fast-paced development. This ensures **stability and reliability** of the voice agents even as we iterate rapidly.

Multi-Agent Squad Design (Voice Flow)

One of our platform's strengths is the **multi-agent squad** design: instead of one monolithic AI trying to handle everything, we deploy a team of specialized assistants that can hand off to each other seamlessly

¹ . Here's our plan for the squad's structure and call flow in the hospitality scenario:

- **Squad Structure:** We will create a Squad comprising four AI assistants (members), each with a defined role: **Concierge**, **Booking Assistant**, **Customer Experience (Support)**, and **Virtual General Manager**. The Concierge is the initial agent (entry point of all calls). Each assistant will have configured **transfer destinations** for certain triggers – for example, the Concierge has destinations for “Booking” and “Support” flows. If a transfer trigger is met, Vapi will switch the active assistant mid-call, carrying over context. The transfer can be accompanied by a brief on-hold message or a phrase like “Please hold while I transfer you to our booking specialist.” (We will use **silent transfers** for data-only handoff when possible, so the user might not even realize a different bot persona has taken over.)
- **Conversation Flow:** A typical call might go as follows: A guest calls the hotel's number. The **Concierge AI** answers with a greeting, verifies the caller's identity (Feature #2) and identifies intent. If the caller wants to make a new reservation, Concierge triggers a transfer to the **Booking AI** ¹ . The Booking assistant then handles the room search, pricing, and booking confirmation using Cloudbeds tools. If the caller instead had a complaint about their stay, Concierge would transfer to the **Customer Experience AI** (which is trained to handle issues, apologies, maybe offer compensation via a Cloudbeds API call like issuing a discount or recording a ticket). If the caller says something that requires managerial approval (“I want a full refund” or “I need to speak to a manager”), the Customer Experience agent can escalate to the **Virtual General Manager AI**. The Virtual GM acts with higher authority – it can handle VIP requests, override policies within limits, or at least gather all info and promise human follow-up.
- **Context Persistence:** Vapi's infrastructure ensures that all these assistants in the squad share the conversation history and state up to that point ¹² . We will double-check that key variables and structured data (as discussed in Feature #1 and #5) are indeed accessible to the new assistant after a transfer. We might use the `analysisPlan.structuredDataMultiPlan` so that each assistant's collected data is merged ⁷ . As a backup, our backend can also maintain a session state that any tool can query for the latest info (e.g., the GM agent upon starting could call a “getSessionInfo” tool that our backend serves, returning a summary of what happened).
- **Natural Handoff Experience:** We will craft the transfer prompts to be smooth. Each assistant's persona is slightly different, but they should still feel like a cohesive team. For example, the voices might differ slightly (maybe Concierge has a friendly female voice, Booking a professional male voice, etc., using different ElevenLabs voice profiles), but the transition will always be explained or obvious (“I'm transferring you now”). If possible, we'll attempt **continuous persona** for minor handoffs – e.g., the user asks a complex billing question that the support agent can handle but it's originally concierge – we might configure the concierge to just use a billing tool without a full transfer. We'll strike a balance between specialization and not over-transferring the caller.

Scalability: The squad design makes it easy to add or remove agents for different industries. For a different vertical, we could swap out the members but keep the same orchestrator logic. It also means each agent's prompt is smaller and more focused, which improves quality and lowers hallucinations (since each has fewer goals) ¹².

Agent Role Definitions and Prompt Design

We will deploy **multiple specialized AI agents**, each with a clear mandate and finely tuned prompt. Below we define each agent's role and highlight key prompt instructions ("system" persona settings) for that agent:

Concierge AI Assistant

Role/Purpose: First point of contact for all calls – think of this as the hotel front desk operator. The Concierge handles greetings, identity verification, and intent routing. This agent's main job is to figure out what the caller needs and either handle simple requests or hand off to the appropriate specialist agent. It maintains a friendly, welcoming tone, embodying the hotel's hospitality ethos.

Prompt & Behavior: The Concierge's system prompt will establish it as a friendly hotel receptionist. For example: *"You are an AI Concierge for [Hotel Name]. Greet callers warmly, gather their name and reason for calling. Determine if they need a new reservation, help with an existing booking, general info, or have an issue/complaint. Verify their identity if discussing personal reservations (use the verify-code tool if needed). If the request is about new bookings, use the transfer to the 'Booking Assistant'. If it's about an existing reservation or issue, transfer to 'Customer Support'. Always confirm you are transferring them and introduce the next assistant."* The prompt will also instruct on using tools: e.g., *"If the caller provides a name or phone, use the 'lookupGuest' tool to find their profile in the system."* The Concierge agent thus gathers as much context as possible (setting global variables like `guestName`, `reservationId`, etc.) before transferring. Simplicity and accuracy are key – it should not attempt to handle detailed tasks beyond initial Q&A.

Booking AI Assistant

Role/Purpose: This agent functions as the **Booking Specialist**. Once engaged, it takes the caller through the process of checking room availability, pricing, and making or modifying a reservation. It interfaces heavily with the Cloudbeds API (through tools) to get real-time data. The tone is professional, efficient, and sales-friendly (it should upsell politely if appropriate, e.g., mention upgrades).

Prompt & Behavior: The Booking assistant's prompt defines it as an expert in reservations. For example: *"You are a Booking Agent AI for a hotel, helping guests book rooms or modify their bookings. You have access to real-time hotel inventory and pricing via tools. When the user asks for availability or to make a booking, use the 'searchAvailability' tool to find open rooms ¹⁶. Always confirm dates, number of guests, and preferences. Once the guest chooses a room, use the 'createReservation' tool to book it, then confirm the reservation number to the caller. If modifying or cancelling, verify details then call the appropriate tool (modifyReservation, cancelReservation)." We'll also include fallback behavior: "If inventory is not available for requested dates, suggest alternative dates or waitlist."* The prompt ensures the agent cites data from the API rather than guessing (e.g., *"Use the tools rather than making up info."*). After completing its task, if the conversation is meant to continue (e.g. user has another question), the Booking agent can either handle it if simple or transfer back to Concierge or to Support as needed.

Customer Experience AI (Support)

Role/Purpose: The **Customer Experience** agent is essentially the support/helpdesk persona. It handles complaints, service issues, questions about the stay, or general inquiries (like “What time is breakfast?” or “Can I get a late checkout?”). It needs to be empathetic, apologetic when needed, and resourceful in resolving problems. It can also handle things like recording a service request (maintenance needed in a room) or answering FAQs.

Prompt & Behavior: The prompt will cast it as a caring customer support agent: *“You are a Customer Experience AI for the hotel, focused on ensuring guest satisfaction. Handle any issues or questions the guest has. Apologize sincerely for any inconvenience and offer to help. You have tools available: for example, ‘logTicket’ to create a support ticket, ‘adjustReservation’ to grant a late checkout or change a booking, and access to hotel info (check-out times, amenities hours, etc.). If the guest asks a common question (like amenities or location), answer from your knowledge base. If it’s a complaint (room not clean, etc.), apologize and either offer a solution (send housekeeping, tool: `createTaskHousekeeping`) or escalate to the manager if beyond your authority.”* The agent is instructed on escalation criteria: *“If the guest demands something outside of policy or is very upset, transfer to the Virtual GM assistant with a briefing.”* In its prompt, we emphasize empathy: *“Always acknowledge the guest’s feelings and reassure them you will do your best.”* This agent thus acts as the voice of customer care, using data and tools to solve issues on the spot when possible (for instance, waiving a fee via Cloudbeds if we provide such a tool).

Virtual General Manager AI

Role/Purpose: The **Virtual GM** is the highest-level assistant, reserved for complex or sensitive situations. This agent represents a manager or someone with authority. It may take over calls that need special handling – e.g., VIP customers, serious complaints, or out-of-ordinary requests. The Virtual GM’s style is calm, confident, and empowered to make decisions (within some limits). It should make the caller feel heard and valued, and it can either resolve the issue or assure follow-up by a human manager if needed.

Prompt & Behavior: We will define the Virtual GM prompt to position it as the final escalation point: *“You are the Virtual General Manager AI. Only calls that are escalated reach you – typically complex issues or VIP requests. Your goal is to make the guest satisfied. You have broad authority to make exceptions, offer compensations (use the ‘offerCompensation’ tool to give a discount or voucher), and authorize actions. Greet the caller by acknowledging you are a manager and you’re here to help. Listen carefully to their concerns (the previous assistant will have transferred you with context of the issue). Provide a solution or plan: whether it’s a refund, a personal apology, or assurance of corrective action. If something truly requires human intervention (e.g. a legal issue or something the AI can’t handle), apologize and promise the real GM will follow up, then ensure a human gets a detailed report.”* The Virtual GM prompt also includes: *“Coordinate with the other systems: you can use tools to adjust reservations, issue refunds (within a limit), or record a detailed incident report (tool: `logIncident`).”* Essentially, this agent ties up loose ends. Even though it’s AI, it should convincingly project the authority of a hotel manager. After handling the matter, it will politely conclude the call.

Each agent’s prompt will be thoroughly tested and iterated to strike the right tone and ensure the correct triggers for tool usage and transfers. By **compartmentalizing prompts** per agent, we greatly reduce complexity vs one giant prompt, and allow focused optimization for each task.

Technology Stack & Tools Summary

Our solution brings together several technologies, each chosen for speed and capability:

- **Vapi Voice AI Platform:** Provides core voice AI functionality – telephony integration, voice transcription & streaming, conversation management, multi-assistant squads, and tool-calling APIs. Vapi's platform handles scalability (99.99% uptime, low latency) and cross-language support out of the box ¹⁷ ⁴. This lets us focus on conversation design rather than low-level telephony.
- **OpenAI LLM:** The intelligence behind our agents is OpenAI's GPT model (likely GPT-4 for its advanced understanding). We use it via Vapi's integration (providing our API key) ⁴. All agent prompts and responses are generated by this model, giving us state-of-the-art language capabilities without training our own model initially.
- **ElevenLabs TTS:** We integrate ElevenLabs for high-quality text-to-speech voices. Vapi supports custom TTS providers ⁴; by plugging in ElevenLabs API, our AI agents will speak in natural, expressive voices that align with the brand (we can choose a voice profile for each persona). This greatly enhances caller experience compared to stock voices.
- **Node.js Backend:** Our custom server (Node.js) glues everything together. It will run our Vapi CLI scripts, serve API endpoints for custom tools, handle Cloudbeds API calls, and implement any custom logic (like verification code generation, complex data processing). Node was chosen for its rich ecosystem and our team's familiarity, plus the async I/O model suits handling webhook calls efficiently.
- **Supabase:** Serves as our backend cloud platform – its **Edge Functions** give us a serverless environment geographically close to users (for low latency on voice webhooks), and its Postgres database stores persistent data (user profiles, logs, etc.). We benefit from Supabase Auth for easily sending magic link emails and possibly managing any user accounts (if down the line we have businesses logging into our platform). Supabase's real-time capabilities could even let us monitor calls live or update dashboards as calls happen. All with minimal DevOps overhead.
- **ShadCN UI & Tailwind CSS:** Our front-end stack for building any web interface needed (admin dashboard, web forms for users). ShadCN UI is a collection of beautifully designed React components that pair with Tailwind CSS utility classes. This choice allows us to spin up interfaces quickly without spending time on design from scratch. Tailwind ensures a consistent design language and responsive layout. We'll use these tools to create a cohesive user experience, whether the user is an internal admin looking at call stats or a hotel guest filling a form on their phone.
- **Cloudbeds API:** While not a technology we build, it's a core external component. Cloudbeds gives us a wide range of hotel management capabilities programmatically – from reservations to guest notes to payments. We will treat Cloudbeds as a critical data source and sink. All interactions with it are through secure API calls, abstracted by our integration layer. In the future, we can swap or add other integrations (for example, connecting to a CRM or another PMS) with the same pattern we use for Cloudbeds.
- **Other Integrations:** By using Vapi, we keep the door open to integrate other services easily. Vapi has integrations to things like calendars, CRMs, payment gateways, etc. ¹⁸ ¹⁹. For example, we might integrate an email API to send follow-up summaries to guests, or a CRM tool to log call outcomes. Our architecture is modular enough to incorporate these as needed (many can be done via Vapi's pre-built **Make.com** integration tools ²⁰ or via custom webhooks similar to Cloudbeds tools).

In summary, our tech stack is modern, robust, and largely composed of **off-the-shelf components** that are **glued together strategically**. This allows rapid development while leveraging proven systems for each concern.

Development Timeline & Milestones

Time is of the essence – we aim to have a functional MVP by **November 12, 2025**. Below is a rough timeline with key milestones to ensure we hit that target:

- **Week 1 (Oct 30 – Nov 5): Setup & Core Architecture**

- *Infrastructure:* Set up the development environment – sign up for Vapi platform, obtain API keys (Vapi, OpenAI, ElevenLabs, Supabase). Configure the Vapi CLI locally and connect it to our account. Initialize a Git repository for our configuration and code.
- *Cloudbeds Access:* Register for Cloudbeds developer access (get API keys, Postman collection). Test a couple of API calls via Postman to verify connectivity.
- *Squad & Assistant Stubs:* Using Vapi's dashboard or CLI, create placeholder assistants for Concierge, Booking, Support, GM – with minimal prompts. Create an initial Squad that includes them and assign a test phone number. Perform a **hello world call** to ensure the call flow reaches our system (even if the assistants just say a basic greeting). This tests our telephony integration early.
- *Supabase Backend:* Initialize Supabase project – set up the database schema for any needed tables (e.g. `users`, `calls`, `verifications`). Deploy a simple Edge Function (hello world) and configure the URL. Test calling it via HTTP. This ensures our webhook mechanism is working.
- *Basic UI Skeleton:* Bootstrap a Next.js (or similar) project for our web UI. Implement a very simple page that can be served for web links (just to verify our ability to send a link). Not critical for MVP, but starting early to integrate the pieces.

- **Week 2 (Nov 6 – Nov 12): Feature Implementation & Testing**

- *Customer Identification & Verification:* Implement the phone/email verification flow. This involves: writing a Supabase function to send an SMS (we might integrate with Twilio's API for SMS since Supabase can call external URLs) and verify codes; updating the Concierge prompt to invoke this process. Test by calling and going through a fake verification. Also code the Cloudbeds lookup on call start: e.g., an Edge Function that given a phone number returns guest info. Incorporate it as a tool and ensure Concierge uses it to personalize greeting.
- *Cloudbeds Tool Integration:* Write a script to generate the JSON for Cloudbeds-related tools (at least the critical ones for booking flow). Use Vapi CLI to create these tools on our account. Develop the corresponding Supabase functions for each, which take the tool inputs, call Cloudbeds API, and return a result. Focus on 2–3 key flows: (a) **Search availability** (dates -> available room types), (b) **Create reservation**, (c) **Retrieve reservation details** (for modifications), and (d) **Cancel reservation**. Additionally, support tools for reading data (guest profile, etc.). Once done, update the Booking assistant's prompt with instructions to use these. Integration test: simulate a booking conversation end-to-end internally and see if the tools return correct data (e.g. we might log tool responses).
- *Web Link & Form Handling:* Build out the form page template using ShadCN components. Implement one example: a "Credit Card Authorization" form or a "Feedback form" that the support agent might send. The form submission should hit a Supabase function that then triggers a call status update.

Perhaps use Vapi's call webhook to inform the agent, or have the agent poll via a short-interval tool ("checkFormStatus"). Test this by simulating a user clicking the link while on a call (we may do this manually for now by copying the link from logs). Adjust the Support agent prompt to send that link at an appropriate moment.

- *Context & Prompt Refinements*: Populate each agent's system prompt with more detailed instructions (as outlined in the prompts section). Include variable placeholders and ensure they align with the data our tools provide. For example, if our availability tool returns a list of rooms, the Booking agent prompt should explain how to present that (maybe reading out a couple of options and prices). Run a **few sample dialogues** in text form to see if the prompts yield the desired behavior (using Vapi's testing or even directly querying GPT-4 with our prompt to preview responses). Iterate on wording to fix any misunderstanding.
- *End-to-End Testing*: By mid-week 2, make test calls covering each major path: new booking, existing reservation inquiry, complaint escalation. Have team members play the "caller" role and interact with the system. We will likely uncover bugs (tool not called when expected, voice recognition hiccups, etc.) – fix those promptly. Pay attention to the conversation quality: adjust prompt tone if needed, add fail-safes (like if LLM output doesn't call a tool when it should, maybe enforce via few-shot examples in prompt).
- *Final Tweaks & Launch Prep*: As Nov 12 approaches, ensure all pieces are deployed in "production" mode: e.g., switch any test API keys to live ones, get a dedicated phone number for the service on Vapi (or Twilio SIP trunk if needed), and ensure compliance (like having a brief initial disclaimer if required for call recording/AI usage). We'll also implement logging: configure Vapi's webhook to send call transcripts or structured data at end of call to a Supabase function that stores it (for QA and analytics) ⁷. On the UI side, we might not have a full admin portal yet, but at least a basic display of recent calls and their outcomes could be helpful – we can build that if time permits.
- **Launch (Nov 12)**: Go live with the platform for a pilot client (e.g., one hotel property using Cloudbeds). Closely monitor calls on launch day via logs and perhaps join the calls silently if possible. Any critical issues, use our backup plan – e.g., route calls to a human or a simple voicemail if the AI fails (we can configure Vapi to transfer to a human line on certain fallback conditions). Collect feedback and prepare a post-launch plan for improvements.

Post-MVP Beyond Nov 12: Following the initial launch, we will likely iterate on voice agent behavior (fine-tuning prompts, adding more Cloudbeds endpoints like payments or tasks, etc.). We'll also flesh out the **web dashboard** for our platform (so business owners can configure their voice agents or view analytics), using the groundwork laid with ShadCN UI. Another near-term goal would be to generalize the platform to other industries by swapping out the Cloudbeds integration with another (leveraging the same *Strate* of platform economics – reusing what we built and only replacing the integration layer).

Conclusion

In this plan we detailed how **platformeconomics.ai** will be built from the ground up in a clean, efficient manner. By integrating existing solutions (Vapi for voice, Cloudbeds for hotel data, supabase for backend, etc.) and focusing our development on the **value-add logic** – identification, verification, context, and multi-agent orchestration – we dramatically accelerate our time to market. This technical architecture is designed to deliver a **seamless voice assistant experience** that feels human-like and robust in handling real

business tasks. We have outlined the roles of each AI agent, the flow of information among them, and the concrete steps to implement and deploy the system by our target date.

Armed with this plan, our team will proceed to implementation immediately, confident that this approach will not only meet the November 12 deadline but also lay a solid foundation for scaling to many clients and use-cases beyond. We're poised to transform voice AI in hospitality (and eventually other industries), taking it to the next level with a **platform strategy** that leverages the best tools available ²¹. The end result: a cutting-edge voice agent platform that delights customers, streamlines operations, and generates new revenue opportunities for businesses – all in a matter of weeks from concept to reality.

¹ ¹² ¹⁵ Introduction to Squads (Multi-Assistant Conversations) | Vapi

<https://docs.vapi.ai/squads>

² ⁵ ¹⁸ ¹⁹ ²⁰ Introduction to Tools | Vapi

<https://docs.vapi.ai/tools>

³ ⁹ Introduction

<https://developers.cloudbeds.com/reference/about-pms-api>

⁴ ¹⁴ ¹⁷ ²¹ Vapi - Build Advanced Voice AI Agents

<https://vapi.ai/>

⁶ ¹⁶ How I Created an Appointment Setter AI Voice Agent using Vapi AI, Make.com, and OpenAI | by Aiza Rashid | Medium

<https://medium.com/@aizarashid17/how-i-created-an-appointment-setter-ai-voice-agent-using-vapi-ai-make-com-and-openai-6968ed631499>

⁷ Squads and Structured Data - VAPI

<https://vapi.ai/community/m/1402726071857582110>

⁸ Variables | Vapi

<https://docs.vapi.ai/assistants/dynamic-variables>

¹⁰ ¹¹ ¹³ Using the Query Tool for Knowledge Bases | Vapi

<https://docs.vapi.ai/knowledge-base/using-query-tool>